

BÁO CÁO BIG DATA

Hệ Thống Phát Hiện Thao Túng Thị Trường

Thành viên: Nguyễn Quang Sang, Nguyễn Đình Phú

Mục lục

1	Tổng Quan Dự Án	3
1.1	Bối cảnh	3
1.2	Mục tiêu	3
2	Kiến trúc hệ thống và dữ liệu lựa chọn	4
2.1	Luồng xử lý tổng quát	4
2.2	Các thành phần chính	4
2.3	Dữ Liệu Và Đặc Trưng	5
2.3.1	Nguồn dữ liệu	5
2.3.2	Bộ đặc trưng Machine Learning	5
3	Huấn Luyện Machine Learning	6
3.1	Random Forest bằng scikit-learn	6
3.2	Random Forest bằng Spark MLlib	6
4	Machine Learning Fast Filter	8
4.1	Đặc trưng đầu vào	8
5	Giải thích bài tập	9
5.1	Giải thích thuật toán	9
5.2	Alpha-Beta Search	9
5.3	Hàm đánh giá rủi ro	9
5.4	Đặc tả tìm kiếm Alpha-Beta	9
5.5	Spark RDD và Spark SQL	10
5.6	Phát hiện wash trading	10
5.7	Truy vết tài khoản nghi vấn	10
5.8	Lý do lựa chọn Spark và Redis	11
6	Graph Analytics và Shapley Value	12
6.1	Phát hiện wash trading	12
6.2	Shapley Value	12
7	Kết Quả Demo Tính	13
7.1	Output	13
7.2	Kết luận của lần chạy	13

8	Hướng Dẫn Chạy	14
8.1	Yêu cầu môi trường	14
8.2	Cài thư viện	14
8.3	Build C++ extension	14
8.4	Chạy demo tĩnh	14
8.5	Train Spark ML	14
8.6	Chạy streaming/replay	15
9	Đánh Giá Và Hạn Chế	16
9.1	Điểm mạnh	16
9.2	Hạn chế	16
9.3	Hướng phát triển	16

1. Tổng Quan Dự Án

1.1 Bối cảnh

Thị trường tài chính hiện đại tạo ra lượng dữ liệu số lệnh rất lớn trong thời gian ngắn. Các hành vi như spoofing, layering hoặc wash trading có thể xuất hiện trong vài giây, làm sai lệch tín hiệu cung cầu và gây rủi ro cho nhà đầu tư. Vì vậy, hệ thống giám sát cần vừa xử lý được dữ liệu gần thời gian thực, vừa có khả năng giải thích kết quả để phục vụ phân tích và cung cấp bằng chứng cho các nghiệp vụ kiểm tra đi kèm.

Dự án xây dựng một pipeline phát hiện thao túng thị trường theo hướng lai ghép. C++ được dùng cho lõi Limit Order Book, Python điều phối pipeline, scikit-learn và Spark MLlib huấn luyện bộ lọc Machine Learning, Apache Spark phân tán quá trình phân tích kịch bản, NetworkX phát hiện cộng đồng giao dịch chéo và Shapley Value xác định tài khoản có vai trò trung tâm trong cụm nghi vấn.

1.2 Mục tiêu

- Mô phỏng và cập nhật trạng thái Limit Order Book với hiệu năng cao
- Sử dụng thông tin số lệnh để phát hiện dấu hiệu spoofing
- Huấn luyện mô hình Random Forest nhẹ bằng scikit-learn và Spark MLlib để làm một lớp lọc ban đầu
- Phân tán phân tích nhiều kịch bản rủi ro bằng Spark RDD và tổng hợp bằng Spark SQL
- Phát hiện cụm wash trading bằng đồ thị có hướng
- Tính Shapley Value để xác định tài khoản chủ mưu
- Lưu kết quả chạy thành JSON/CSV

2. Kiến trúc hệ thống và dữ liệu lựa chọn

2.1 Luồng xử lý tổng quát

```
Replay / Static Demo / Binance Stream
  |
  v
Redis Stream / Internal Simulated Data
  |
  v
Python Orchestrator
  |
  +-- Layer 1: ML Fast Filter
  +-- Layer 2: C++ Limit Order Book + Alpha-Beta Search
  +-- Layer 3: Spark RDD + Spark SQL
  +-- Layer 4: Graph Analytics + Shapley Value
  |
  v
Output: JSON / CSV
```

2.2 Các thành phần chính

Thành phần	File chính	Vai trò
C++ Limit Order Book	src/cpp/lob.cpp, src/cpp/lob.h	Quản lý bid/ask, thêm lệnh, khớp lệnh, hủy lệnh, tính trạng thái thị trường cũng như các thuật toán duyệt cây.
Python binding	src/cpp/bindings.cpp	Đóng gói C++ thành module Python <code>lob_core</code> bằng <code>pybind11</code> .
ML fast filter	src/python/ml_filter.py	Trích xuất đặc trưng và dự đoán xác suất spoofing.
Train sklearn model	src/python/train_ml_model.py	Huấn luyện Random Forest bằng <code>scikit-learn</code> trên feature CSV.
Train Spark ML model	src/python/train_spark_ml_model.py	Huấn luyện RandomForestClassifier bằng Spark MLlib.
Search engine	src/python/search_engine.py	Phân tích kịch bản bằng Alpha-Beta và heuristic risk score.
Spark engine	src/python/spark_engine.py	Phân tán các kịch bản rủi ro và tổng hợp thống kê.
Replay engine	src/python/replay_manipulation.py	Đẩy dữ liệu replay vào Redis Stream để kiểm thử streaming.
Streaming engine	src/python/streaming_engine.py	Đọc Redis Stream bằng Spark Structured Streaming và phát hiện cảnh báo.
Graph/Shapley	src/python/shapley_analyzer.py	Phát hiện cộng đồng wash trading và tính Shapley Value.
Output recorder	src/python/output_recorder.py	Lưu JSON/CSV theo từng lần chạy.

2.3 Dữ Liệu Và Đặc Trưng

2.3.1 Nguồn dữ liệu

Do dữ liệu sổ lệnh thực tế thường khó tiếp cận hoặc cần trả phí như LOBSTER, bài tập sử dụng FI-2010 làm dữ liệu nền cho Limit Order Book. Từ dữ liệu này, nhóm chuyển đổi sang replay JSON và chèn thêm các spoofing episode để tạo dữ liệu có nhân phục vụ huấn luyện, kiểm thử và demo streaming. Riêng phần wash trading được mô phỏng bằng dữ liệu giao dịch giữa các tài khoản để minh họa module graph analytics và Shapley Value, do FI-2010 không chứa thông tin định danh tài khoản buyer/seller. Các luồng dữ liệu đầu vào như sau:

- Dữ liệu replay JSON dùng cho streaming/replay.
- Dữ liệu FI-2010 đã chuyển thành feature CSV để huấn luyện Machine Learning.
- Dữ liệu giao dịch tài khoản mô phỏng dùng cho module wash trading và Shapley Value.

2.3.2 Bộ đặc trưng Machine Learning

Mô hình ML sử dụng cùng một tập đặc trưng:

Đặc trưng	Ý nghĩa
spread	Khoảng cách giữa best ask và best bid.
bid_vol	Tổng khối lượng phía mua trong sổ lệnh.
ask_vol	Tổng khối lượng phía bán trong sổ lệnh.
imbalance	Mức mất cân bằng thanh khoản giữa bid và ask.
cancel_rate	Tỷ lệ hủy lệnh, dùng để nhận diện hành vi đặt rồi hủy nhanh.

3. Huấn Luyện Machine Learning

3.1 Random Forest bằng scikit-learn

Bài tập thử nghiệm 2 hướng, với hướng đầu tiên là Random Forest bằng scikit-learn. Mô hình được lưu tại `models/spoofing_rf_model.pkl`. Kết quả thực nghiệm được ghi trong `outputs/ml_fi2010_metrics.json`.

Thông số	Giá trị
Loại mô hình	RandomForestClassifier
Số cây	200
Độ sâu tối đa	10
Class weight	balanced_subsample
Train samples	50,000
Test samples	10,000
Tỷ lệ hành vi spoofing ở train	3.006%
Tỷ lệ hành vi spoofing ở test	2.4%

Metric	Kết quả
Accuracy	0.8461
Precision	0.1177
Recall	0.8333
Positive F1	0.2063
ROC-AUC	0.9120
Confusion matrix	[[8261, 1499], [40, 200]]
Thời gian train	3.9s

Kết quả cho thấy accuracy cao chịu ảnh hưởng lớn từ tình trạng mất cân bằng dữ liệu, vì số mẫu bình thường nhiều hơn đáng kể so với mẫu spoofing. Do đó, precision, recall, F1-score và ROC-AUC là các chỉ số cần theo dõi cùng với accuracy. ROC-AUC khoảng 0.81 cho thấy mô hình vẫn có khả năng phân tách tín hiệu spoofing tốt hơn mức ngẫu nhiên, nhưng precision và recall cho thấy bài toán phát hiện lớp hiếm vẫn còn khó và cần tiếp tục cải thiện.

3.2 Random Forest bằng Spark MLlib

Spark ML được bổ sung để chứng minh khả năng huấn luyện mô hình trên framework Big Data thay vì chỉ dùng scikit-learn local. Script huấn luyện là `src/python/train_spark_ml_model.py`. Mô hình sử dụng `pyspark.ml.classification.RandomForestClassifier`,

`VectorAssembler` để tạo cột `features`, và `class_weight` để giảm ảnh hưởng mất cân bằng nhãn.

Lần chạy được ghi trong metrics sử dụng 20 cây, độ sâu tối đa 6 và seed 42.

Thông số	Giá trị
Loại mô hình	Spark MLlib RandomForestClassifier
Số cây	20
Độ sâu tối đa	6
Weight column	class_weight
Train label 0 (bình thường)	48,497
Train label 1 (spoofing)	1,503
Thời gian train	4.57 giây

Metric	Kết quả
Accuracy	0.7090
Positive precision	0.0710
Positive recall	0.9208
Positive F1	0.1319
ROC-AUC	0.9107
confusion matrix	[[6869, 2891], [19, 221]]

Với bộ dữ liệu thử nghiệm chỉ khoảng 60,000 dòng và 5 đặc trưng, Spark MLlib chịu thêm chi phí khởi tạo Spark session, phân vùng dữ liệu và quản lý job, nên thời gian chạy có thể chậm hơn scikit-learn local. Tuy nhiên, đối với luồng dữ liệu số lệnh có quy mô lớn hơn, cơ chế xử lý song song và phân tán của Spark có thể phát huy lợi thế nhờ khả năng phân vùng dữ liệu ra nhiều worker node và giảm áp lực bộ nhớ trên một máy duy nhất.

4. Machine Learning Fast Filter

4.1 Đặc trưng đầu vào

Tầng ML trích xuất 5 đặc trưng: spread, bid volume, ask volume, imbalance và cancellation rate. Các đặc trưng được dùng để dự đoán xác suất spoofing bằng Random Forest.

Để đảm bảo khả năng tái lập, các đặc trưng được hiểu theo các quy ước sau. Với p_1^{ask} là best ask, p_1^{bid} là best bid, q_i^{bid} và q_i^{ask} là khối lượng ở mức giá thứ i :

$$\text{spread} = p_1^{ask} - p_1^{bid}$$

$$V_b = \sum_{i=1}^L q_i^{bid}, \quad V_a = \sum_{i=1}^L q_i^{ask}$$

$$\text{imbalance} = \frac{V_b - V_a}{V_b + V_a}$$

$$\text{cancel_rate} = \frac{N_{cancel}}{N_{new} + N_{cancel}}$$

Trong demo hiện tại, V_b và V_a được tính trên các level đang có trong snapshot LOB được đưa vào pipeline. imbalance là tỷ lệ có dấu: giá trị dương cho biết phía bid đang chiếm ưu thế về volume, giá trị âm cho biết phía ask chiếm ưu thế. cancel_rate được tính trên cửa sổ sự kiện đang được replay/stream vào trạng thái LOB tại thời điểm đánh giá.

Dữ liệu FI-2010 được chuyển sang replay JSON, sau đó các episode spoofing tổng hợp được chèn vào bằng cách tạo cụm lệnh volume lớn gần best bid/best ask và hủy sau một số event ngắn. Nhân spoofing được gán cho các event nằm trong episode chèn thêm. Để tránh rò rỉ dữ liệu, train và test dùng file nguồn khác nhau, tập test được tạo từ file raw riêng, không lấy lại các cửa sổ replay gần trùng với tập train.

5. Giải thích bài tập

5.1 Giải thích thuật toán

Trong `lob.cpp`, giai đoạn Alpha-Beta Search được mô hình hóa theo hướng tìm kiếm trên cây kịch bản. Sau khi lớp ML filter đánh dấu một trạng thái sở lệnh là nghi vấn, hệ thống sinh ra nhiều hành động giả lập như thêm lệnh lớn, hủy lệnh hoặc giữ nguyên trạng thái để đánh giá tác động lên thị trường. Cách tiếp cận này lấy cảm hứng từ game-theoretic search, trong đó hành vi thao túng và phản ứng của thị trường được xem như các nhánh hành động đối lập. Alpha-Beta Search giúp giảm số nhánh cần duyệt khi đánh giá các kịch bản có rủi ro cao.

5.2 Alpha-Beta Search

Sau khi ML filter đánh dấu một trạng thái có rủi ro, pipeline tạo nhiều kịch bản giả lập quanh trạng thái đó. Search engine đánh giá tác động của các hành động như thêm lệnh lớn, hủy lệnh hoặc giữ nguyên trạng thái. Alpha-Beta Search giúp giảm số nhánh cần duyệt, còn heuristic score đánh giá các yếu tố như spread, imbalance, volume và cancel rate. Quiescence search được dùng để hạn chế việc kết luận nhầm một biến động nhất thời của thị trường thành hành vi thao túng.

5.3 Hàm đánh giá rủi ro

Hàm heuristic kết hợp spread, imbalance và cancellation rate:

$$\text{Risk} = \text{Spread} \times 10 + \text{Imbalance} \times 50 + \text{CancellationRate} \times 30$$

Nếu sở lệnh rơi vào trạng thái crossed book, hệ thống gán điểm rủi ro rất cao để phản ánh bất thường nghiêm trọng.

5.4 Đặc tả tìm kiếm Alpha-Beta

Trong phạm vi project, `alpha_beta_search` được dùng như một lớp heuristic search để đánh giá các trạng thái LOB sinh ra từ kịch bản mô phỏng. Trạng thái s gồm tập lệnh bid/ask hiện tại, best bid, best ask, spread, imbalance và cancellation rate. Hàm tiện ích cuối cùng là:

$$U(s) = \text{Risk}(s)$$

với $\text{Risk}(s)$ được tính bởi công thức heuristic ở trên. Các hành động được xét trong mô phỏng gồm thêm lệnh, hủy lệnh và thực hiện market order với volume thay đổi theo từng scenario. Ở tầng Spark, mỗi scenario được xem là một nhánh độc lập, Alpha-Beta Search chỉ

dùng để lan truyền điểm heuristic trong cây trạng thái nhỏ của từng scenario, không dùng để chứng minh chiến lược tối ưu tuyệt đối của một tác nhân thao túng.

Mô tả rút gọn:

```
alpha_beta(state, depth, alpha, beta, maximizing):
    if depth == 0 or terminal(state):
        return evaluate_state(state)

    if maximizing:
        value = -inf
        for action in legal_actions(state):
            value = max(value, alpha_beta(apply(state, action),
                                           depth - 1, alpha, beta, false))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value

    value = inf
    for action in legal_actions(state):
        value = min(value, alpha_beta(apply(state, action),
                                       depth - 1, alpha, beta, true))
        beta = min(beta, value)
        if alpha >= beta:
            break
    return value
```

5.5 Spark RDD và Spark SQL

Các kịch bản được phân tán bằng Spark RDD để chạy song song trên nhiều partition. Mỗi kịch bản trả về một heuristic risk score. Sau đó, kết quả được chuyển thành DataFrame để Spark SQL tổng hợp các thống kê như giá trị trung bình, lớn nhất, nhỏ nhất, độ lệch chuẩn và percentile. Cách làm này tách rõ hai nhiệm vụ: RDD dùng để phân tán tính toán kịch bản, còn Spark SQL dùng để tổng hợp và phân tích kết quả.

5.6 Phát hiện wash trading

Dự án mô hình hóa giao dịch giữa các tài khoản thành đồ thị có hướng. Mỗi node là một tài khoản, mỗi cạnh là dòng giao dịch từ buyer sang seller. NetworkX tìm các vùng liên thông mạnh để phát hiện nhóm tài khoản có giao dịch chéo lặp đi lặp lại, đây là dấu hiệu thường gặp của wash trading.

5.7 Truy vết tài khoản nghi vấn

Sau khi có cộng đồng nghi vấn, hệ thống dùng Monte Carlo Shapley Value để ước lượng đóng góp của từng tài khoản vào tổng thanh khoản nội bộ. Tài khoản có Shapley Value cao nhất được xem là ứng viên ringleader.

5.8 Lý do lựa chọn Spark và Redis

Đối với thị trường có số lượng cập nhật số lệnh rất lớn, một kiến trúc xử lý tuần tự dễ gặp bottleneck khi phải đánh giá nhiều kịch bản rủi ro. Spark được sử dụng để phân tán quá trình tính toán heuristic và chạy các giả lập kịch bản giao dịch. Sau khi được lọc bởi lớp Random Forest, các trạng thái nghi vấn được chuyển sang lớp phân tích sâu hơn bằng C++ và Spark.

Trong phạm vi project, Redis Stream được chọn làm message buffer nhẹ cho demo streaming vì dễ triển khai trên môi trường local và thao tác đọc/ghi nhanh. Kafka phù hợp hơn với hệ thống production cần log phân tán, khả năng chịu lỗi và mở rộng lớn, nhưng sẽ làm tăng độ phức tạp triển khai trong phạm vi bài tập. Do đó, pipeline streaming hiện tại sử dụng Redis Stream để kết nối replay data với Spark Structured Streaming.

6. Graph Analytics và Shapley Value

6.1 Phát hiện wash trading

Các giao dịch tài khoản được biểu diễn thành đồ thị có hướng bằng NetworkX:

$$G = (V, E, w, t)$$

trong đó V là tập tài khoản, E là giao dịch có hướng từ tài khoản bán sang tài khoản mua, w là volume giao dịch và t là thời điểm hoặc thứ tự event. Hệ thống tìm các strongly connected components để phát hiện cụm giao dịch chéo. Kết quả này chỉ cho thấy các tài khoản có quan hệ giao dịch hai chiều hoặc theo chu trình trong cùng cụm, nó chưa tự chứng minh wash trading, ý định thao túng hoặc quyền kiểm soát chung. Vì vậy, cụm được gắn nhãn là nhóm nghi vấn và cần được kiểm tra cùng volume, tần suất, timing và ngữ cảnh giao dịch.

6.2 Shapley Value

Sau khi có community nghi vấn, Spark RDD phân tán nhiều hoán vị Monte Carlo để ước lượng Shapley Value. Hàm đặc trưng $v(S)$ của một coalition S được hiểu là tổng đóng góp thanh khoản/giao dịch nội bộ mà nhóm S tạo ra trong community. Với một tài khoản i , Shapley Value ước lượng đóng góp biên trung bình của i khi được thêm vào các liên minh khác nhau:

$$\phi_i(v) = \frac{1}{M} \sum_{m=1}^M [v(S_m \cup \{i\}) - v(S_m)]$$

trong đó M là số lượng hoán vị Monte Carlo ngẫu nhiên được chọn, và S_m là tập các tài khoản đứng trước tài khoản i trong hoán vị thứ m .

7. Kết Quả Demo Tính

7.1 Output

Pipeline tính lưu kết quả vào thư mục `outputs/<run_id>/`. Một lần chạy gần nhất có `run_id` là 20260621_153840. Các artifact chính gồm:

- `01_initial_lob_snapshot.json`: trạng thái LOB ban đầu.
- `02_ml_filter_result.json`: xác suất spoofing từ ML filter.
- `03_scenario_results.csv`: kết quả từng kịch bản.
- `04_spark_summary.json`: thống kê tổng hợp từ Spark.
- `05_top5_scenarios.json/csv`: top kịch bản rủi ro.
- `06_wash_trading_community.json`: cụm giao dịch chéo nghi vấn.
- `07_shapley_values.json/csv`: giá trị Shapley của các tài khoản.
- `10_final_result.json`: kết luận cuối cùng của pipeline.

7.2 Kết luận của lần chạy

Kết quả cuối cùng trong `10_final_result.json`:

Trường	Giá trị
Status	SPOOFING_AND_WASH_TRADING
Severity	CRITICAL
Ringleader	ACC_RINGLEADER
Community	ACC_1, ACC_2, ACC_3, ACC_RINGLEADER
Average risk	54.31
Maximum risk	78.91
Spoofing probability	1.0

Kết quả này cho thấy pipeline đã đi hết toàn bộ chuỗi xử lý: tạo trạng thái LOB, chạy ML filter, phân tích kịch bản bằng Spark, phát hiện cộng đồng wash trading và tính Shapley Value để xác định ringleader.

8. Hướng Dẫn Chạy

8.1 Yêu cầu môi trường

Trên Windows:

- Python 3.11 64-bit.
- Java 17 cho PySpark.
- CMake và MSVC Build Tools để build C++ extension.
- Redis hoặc Docker Desktop nếu chạy streaming/replay.

8.2 Cài thư viện

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 -m pip install -r requirements.txt
```

8.3 Build C++ extension

Python không import trực tiếp file C++, cần build lỗi C++ thành extension Python tương thích với môi trường chạy.

Trên Windows:

```
cd path\to\BigDataSang
$env:PYTHON_EXE=(py -3.11 -c "import sys; print(sys.executable)")
cmake -S . -B build-py311-v2 -G "NMake Makefiles" -DCMAKE_POLICY_VERSION_MINIMUM=3.5 -
DPYTHON_EXECUTABLE="$env:PYTHON_EXE"
cmake --build build-py311-v2
```

8.4 Chạy demo tĩnh

Trên Windows:

```
cd path\to\BigDataSang
$env:PYTHONIOENCODING="utf-8"
$env:PYSARK_PYTHON=(py -3.11 -c "import sys; print(sys.executable)")
$env:PYSARK_DRIVER_PYTHON=$env:PYSARK_PYTHON
& $env:PYSARK_PYTHON src/python/main.py
```

8.5 Train Spark ML

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 src/python/train_spark_ml_model.py --num-trees 20 --max-depth 6
```

Sau khi chạy, kiểm tra kết quả tại `outputs/spark_ml_fi2010_metrics.json` và `models/spark_ml_rf/`.

8.6 Chạy streaming/replay

Bước 1: Khởi động Redis (Có thể dùng Docker hoặc dịch vụ Redis chạy trực tiếp trên host):

```
docker compose up -d
```

Bước 2: Khởi động Spark streaming engine (Terminal 1)

Trên Windows:

```
cd path\to\BigDataSang
$env:HADOOP_HOME="$PWD\hadoop"
$env:hadoop_home=$env:HADOOP_HOME
$env:Path="$env:HADOOP_HOME\bin;$env:Path"
$env:PYTHONIOENCODING="utf-8"
$env:PYSARK_PYTHON=(py -3.11 -c "import sys; print(sys.executable)")
$env:PYSARK_DRIVER_PYTHON=$env:PYSARK_PYTHON
& $env:PYSARK_PYTHON src/python/streaming_engine.py
```

Bước 3: Đẩy replay data vào Redis Stream (Terminal 2)

Trên Windows:

```
cd path\to\BigDataSang
py -3.11 src/python/replay_manipulation.py --file data/replay/test_spoofing_replay.json --sleep 0.1
```


9. Đánh Giá Và Hạn Chế

9.1 Điểm mạnh

- Kiến trúc lai ghép đầy đủ: C++, Python, Spark, ML, Graph và Shapley Value.
- Có cả demo tĩnh và streaming/replay.
- Kết quả chạy được lưu thành file để kiểm chứng.
- Spark MLlib đã được chạy thành công và sinh model/metrics.

9.2 Hạn chế

- Dataset mất cân bằng mạnh, nên accuracy không đủ để đánh giá riêng lớp spoofing.
- Các hành vi spoofing và wash trading trong thị trường thực tế rất phức tạp, nên hệ thống hiện mới phù hợp ở mức mô phỏng và demo học thuật, chưa đủ điều kiện áp dụng trực tiếp vào thực tiễn.
- Spark phù hợp với batch hoặc near real-time, nhưng chưa phải lựa chọn tối ưu cho các hệ thống pre-trade latency cực thấp.
- Mô hình hiện được huấn luyện trên dữ liệu cố định, nên chưa có cơ chế cập nhật liên tục để thích nghi với các hành vi thao túng mới.

9.3 Hướng phát triển

- Bổ sung các đặc trưng động theo thời gian như rolling imbalance, rolling cancel rate và biến động spread để Random Forest filter phản ánh tốt hơn trạng thái thị trường.
- Tìm kiếm thêm dữ liệu thực tế hoặc benchmark có chất lượng cao để mô hình học được nhiều pattern thao túng hơn.
- Xây dựng dashboard hoặc báo cáo tự động từ các output để hỗ trợ theo dõi kết quả demo.